1. Introduction to PPO and SAC

Proximal Policy Optimization (PPO) [1] and Soft Actor-Critic (SAC) [2] are two widely used deep RL algorithms for continuous control. PPO is on-policy: each update uses trajectories from the current policy, and old data is discarded. SAC is off-policy: it stores transitions in a replay buffer and reuses past experience, making it substantially more sample efficient. PPO seeks the largest policy improvement step without destabilising the policy, replacing the hard KL constraint of TRPO with a clipped surrogate objective.

Before PPO updates the policy, we must estimate how much better each action was compared to the baseline. We implement this via Generalised Advantage Estimation (GAE), which iterates backward through a collected rollout. At each timestep t, we compute the TD error:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t) \tag{1}$$

where d_t is 1 if the episode terminated at step t (masking the bootstrap when there is no valid next state). The advantage is then accumulated via recursively going back the timesteps:

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1} \tag{2}$$

with $\hat{A}_T = 0$ at the rollout boundary. The parameter λ controls the bias-variance tradeoff in advantage estimation: at $\lambda = 0$, estimates reduce to one-step TD errors i.e. low variance but biased because they rely on the value function's accuracy. At $\lambda = 1$, estimation becomes equivalent to Monte Carlo returns, unbiased but high variance due to summing over full trajectories. The return targets used to train the critic are then $R_t = \hat{A}_t + V(s_t)$.

The core update mechanism relies on the importance-sampling ratio $r_{t(\theta)} = \pi_{\theta(a_t|s_t)}/\pi_{\theta_{\text{old}}}(a_t|s_t)$, computed via log-space subtraction then exponentiation for numerical stability. When this ratio is close to 1, the new policy behaves similarly to the old one. PPO prevents it from deviating too far by clipping it:

$$L^{\text{CLIP}}(\theta) = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} \max\Big(-\hat{A}_t \cdot r_t(\theta), -\hat{A}_t \cdot \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\Big) \tag{3}$$

If the advantage is positive the action was good, so we increase its probability, but only up to a factor of $(1 + \epsilon)$; if negative, we clip at $(1 - \epsilon)$. The gradient vanishes once the ratio leaves the trusted interval, preventing catastrophic updates seen with naive policy gradient methods.

The critic $V_\phi(s)$ is trained alongside the policy by minimising MSE against the GAE return targets:

$$L^{\text{VF}}(\phi) = \frac{1}{|\mathcal{B}|}\sum_{t} \big(V_\phi(s_t) - R_t\big)^2 \tag{4}$$

This lets the critic learn a smoother estimate of the expected future return, which in turn improves the quality of the advantage estimates used by the actor.

Soft Actor Critic (SAC) is an off-policy actor-critic algorithm built on the maximum entropy RL framework [2]. SAC augments the reward with an entropy bonus $\alpha H(\pi)$, encouraging the policy to remain stochastic for exploration and robustness. The temperature α trades off reward maximisation against entropy and is tuned automatically (see below). We use an actor π_ϕ and critic Q_ψ (omitting the separate soft value network from the original paper).

For training the Q-function network we use the following loss:

$$\begin{aligned} J_Q(\psi) &= E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\Big(Q_\psi(s_t,a_t) - \hat{Q}(s_t,a_t)\Big)^2\right] \text{ where,} \\ \hat{Q}(s_t,a_t) &= r + \gamma\Big(Q_{\overline{\psi}}(s_{t+1},a') - \alpha\log\pi_\phi(a'|s_{t+1})\Big); a' \sim \pi_\phi(\cdot\,|s_t) \end{aligned} \tag{5}$$

The target weights $\overline{\psi}$ are a Polyak average of the online weights: $\overline{\psi} \leftarrow \tau\psi + (1-\tau)\overline{\psi}$, where τ controls how quickly the target tracks the online network.

We train two critics $\{\psi_1, \psi_2\}$ simultaneously and take $\min_i Q_{\overline{\psi}_i}$ in the target to mitigate overestimation bias.

We train the actor using the following loss:

$$J_\pi(\phi) = E_{a\sim\pi_\phi}\left[\alpha\log\pi_\phi\big(a_\phi|s_t\big) - \min_{i=\{1,2\}}\Big(Q_{\psi_i}\big(a_\phi, s_t\big)\Big)\right] \tag{6}$$

Here we employ the reparameterization trick to reparameterize actions a as $a_\phi(s,\xi)$ where $\xi \sim N(0,I)$ making a_ϕ a deterministic function of ϕ and ξ allowing us to push the gradient through the sampling operation.

Additionally, instead of treating α as a fixed parameter we update it along with the other parameters.

$$J(\alpha) = E_{a\sim\pi_\phi}\Big[-\alpha\Big(\log\pi_\phi(a|s) + \overline{\mathbb{H}}\Big)\Big] \tag{7}$$

This removes α as a manual hyperparameter: it decreases as the policy improves, shifting the agent from exploration to exploitation automatically.

2. Relevance to Robotics

Robotics problems typically involve continuous state and action spaces, noisy observations, and a strong need for stable learning that make classical RL approaches impractical. PPO and SAC address several of these challenges and both natively operate in continuous action spaces: PPO parameterises a Gaussian policy while SAC uses a squashed Gaussian to produce bounded actions, which is fundamental for outputting real-valued torques or joint velocities.

PPO is robust to catastrophic updates due to the clipped objective and is straightforward to implement with a single optimiser, no replay buffer, and no target networks. This makes it easy to parallelise across many simulation instances, making it most suited to sim-to-real pipelines where

data is cheap. SAC is more appealing for real-world robotics where data collection is expensive: it reuses all past experience via a replay buffer for substantially better sample efficiency, and its entropy-regularised objective keeps the policy stochastic, providing inherent robustness to sensor noise.

3. Environment Description

We train in HalfCheetah-v4 and Ant-v4 from MuJoCo Gymnasium [3], [4]. Both are locomotion tasks with continuous action spaces, chosen because they represent different levels of complexity under the same objective, making them directly comparable for benchmarking PPO and SAC.

HalfCheetah-v4 is a planar 2D biped with 6 actuated joints. The state space $\mathcal{S} \subseteq \mathbb{R}^{17}$ contains torso height and pitch, joint angles for all 6 joints, and their time derivatives; the x-position is excluded to prevent the agent from memorising absolute position. The action space $\mathcal{A} \subseteq [-1, 1]^6$ is the torque applied to each joint. The reward is $r_t = w_{\text{forward}} \cdot \dot{x} - w_{\text{ctrl}} \|a_t\|_2^2$, rewarding forward velocity and penalising large torques. The episode never terminates early.

Ant-v4 is a 3D quadruped with 8 actuated joints across 4 legs. The state space $\mathcal{S} \subseteq \mathbb{R}^{27}$ contains torso height, orientation as a quaternion, hip and ankle joint angles for all 4 legs, and their time derivatives. The action space $\mathcal{A} \subseteq [-1, 1]^8$ is the torque on each joint. The reward is $r_t = r_{\text{healthy}} + w_{\text{forward}} \cdot \dot{x} - w_{\text{ctrl}} \|a_t\|_2^2 - w_{\text{contact}} \|F_{\text{contact}}\|_2^2$, adding a per-timestep survival bonus and a contact force penalty. The episode terminates if the torso height leaves $[0.2, 1.0]$.

Notably, HalfCheetah never terminates early while Ant uses health-based termination. This difference significantly affects training dynamics, as discussed in our hyperparameter analysis.

4. PPO Hyperparameter Tuning

We chose to tune clip coefficient (ϵ) and λ (in GAE) as these affect the algorithm's working to its core.

The clip coefficient controls the policy update size. Too small a value slows convergence; too big and the policy overshoots the optimum, both resulting in poor performance. Tuning λ controls the bias-variance tradeoff in advantage estimation. As λ moves from 0 to 1 it causes the shift from TD(0) at $\lambda = 0$ (high bias, bootstraps from the value function) to Monte Carlo at $\lambda = 1$ (unbiased but high variance, accumulated over full episode lengths).

The following values of ϵ were chosen: $\epsilon \in \{0.1, 0.2, 0.3\}$. We have explored values around the published default [1] ($\epsilon = 0.2$) testing change in agent behaviour under a transition from a strict ($\epsilon = 0.1$) to a permissive ($\epsilon = 0.3$) policy update constraint. For λ the following values were chosen: $\lambda \in \{0.9, 0.95, 1.00\}$. A similar strategy of exploring around the published default [1] is followed, testing change in agent behaviour as advantage estimation moves from pure Monte Carlo at $\lambda = 1$ to slightly towards TD(0) at $\lambda = 0.90$.

	$\boldsymbol{\lambda = 0.90}$	$\boldsymbol{\lambda = 0.95}$	$\boldsymbol{\lambda = 1.00}$
$\boldsymbol{\varepsilon = 0.1}$	1471.80	1378.69	1657.81
$\boldsymbol{\varepsilon = 0.2}$	4449.39	2706.83	2397.13
$\boldsymbol{\varepsilon = 0.3}$	1131.88	**4568.06**	150.90

Table 1 : PPO on HalfCheetah-v4: mean episodic return over 10 episodes per (ε, λ) pair

Here we observed that we got the best performance at $\varepsilon = 0.3$ and $\lambda = 0.95$. A permissive policy update strategy works best here because the HalfCheetah doesn't terminate therefore there aren't any catastrophic consequences to making large policy updates. Moreover, $\lambda = 0.95$ works better than full Monte Carlo ($\lambda = 1$) because $\lambda = 0.95$ provides better variance control as it would discount variance just enough to have stable advantage estimates contrary to the full Monte Carlo estimation where it would just accumulate over the entire episode length, additionally it shows resilience towards errors in later rewards whereas in the case of $\lambda = 1$ they would propagate all the way back through the return.

	$\boldsymbol{\lambda = 0.90}$	$\boldsymbol{\lambda = 0.95}$	$\boldsymbol{\lambda = 1.00}$
$\boldsymbol{\epsilon = 0.1}$	2498.17	1419.31	11.57
$\boldsymbol{\epsilon = 0.2}$	2277.72	3049.39	107.12
$\boldsymbol{\epsilon = 0.3}$	**3204.60**	507.76	72.97

Table 2 : PPO on Ant-v4: mean episodic return over 10 episodes per (ε, λ) pair

Here we found $\varepsilon = 0.3$ and $\lambda = 0.90$ showed the best performance. $\varepsilon = 0.3$ still performs the best despite the possibility of destabilisation because under the limited time budget the speed benefit of larger updates outweigh the occasional destabilisation costs, it is a bigger risk to be moving too slowly and not reaching the optimum. Moving further away from Monte Carlo estimation has proven to be beneficial because of the following reasons, firstly, now the survival reward is dense and immediate (the agent receives a +1 reward for every healthy timestep) so short horizon estimation of TD(0) actually captures useful signals, secondly, now the agent is controlling 8 joints across 4 limbs so the estimations are more susceptible to noise therefore having a harder discount improves performance and lastly, with shorter episodes Monte Carlo methods become unreliable since a successful run of 1000 timesteps is not guaranteed. We therefore used ($\epsilon = 0.3, \lambda = 0.95$) for HalfCheetah and ($\epsilon = 0.3, \lambda = 0.9$) for Ant in our final comparison runs.

5. SAC Hyperparameter Tuning

We tuned γ (discount factor, controlling the agent's planning horizon) and τ (Polyak averaging constant, controlling how quickly the target networks track the online networks). We swept $\gamma \in \{0.90, 0.95, 0.99\}$, testing from aggressive discounting to long-horizon planning, and $\tau \in$

$\{0.005, 0.01, 0.05\}$, explored towards the upper range relative to the published default [2], tested on a reduced budget of 450k steps to keep the sweep tractable.

	$\gamma = 0.90$	$\gamma = 0.95$	$\gamma = 0.99$
$\tau = 0.005$	2294.81	2170.51	8556.36
$\tau = 0.01$	2371.91	8707.92	8552.37
$\tau = 0.05$	2385.95	2174.07	**9418.79**

Table 3 : SAC on HalfCheetah-v4: mean episodic return over 10 episodes per (γ, τ) pair

We get the best performance from $\gamma = 0.99$ and $\tau = 0.05$. The domination of $\gamma = 0.99$ is the clearest signal as it outperforms all other values of γ for every value of τ, this is because the locomotion goal that the HalfCheetah-v4 environment poses requires sustained coordinated joint motion over many timesteps i.e. its a long horizon problem hence a high $\gamma = 0.99$ ensures that future rewards are given significant weight. $\tau = 0.05$ dominates because HalfCheetah-v4 has smooth and relatively stationary reward gradients i.e. Q-functions don't change dramatically between updates hence can afford to update more aggressively.

	$\gamma = 0.90$	$\gamma = 0.95$	$\gamma = 0.99$
$\tau = 0.005$	810.21	2272.73	4823.77
$\tau = 0.01$	1102.83	1572.79	**5369.59**
$\tau = 0.05$	809.55	1369.41	3276.02

Table 4 : SAC on Ant-v4: mean episodic return over 10 episodes per (γ, τ) pair

We find that $\gamma = 0.99$ and $\tau = 0.01$ gives the best performance. $\gamma = 0.99$ dominates again for the same reasons discussed previously. However, an interesting thing to notice is that here $\tau = 0.01$ gives better performance as opposed to $\tau = 0.05$ which was best for HalfCheetah. This is because the Q-function landscape now shifts more rapidly, hence aggressive update strategies ($\tau = 0.05$) introduce instability. Early termination causes sharp discontinuities in Q-function values near termination states and a fast moving target network would amplify these rather than smooth them over. Therefore, $\tau = 0.01$ strikes a balance between tracking changes fast enough and remaining conservative enough to avoid instability. We selected ($\gamma = 0.99, \tau = 0.05$) for HalfCheetah and ($\gamma = 0.99, \tau = 0.01$) for Ant in our final comparison runs.

6. Results and Comparison

SAC consistently outperforms PPO across both environments — **9418** vs **4568** on HalfCheetah-v4 and **5369** vs **3204** on Ant-v4 — despite training on fewer than half the timesteps (450k vs 1M), reflecting its substantially greater sample efficiency as an off-policy method. SAC stores transitions in a replay buffer and reuses them across multiple updates, whereas PPO discards rollout data after each on-policy update.

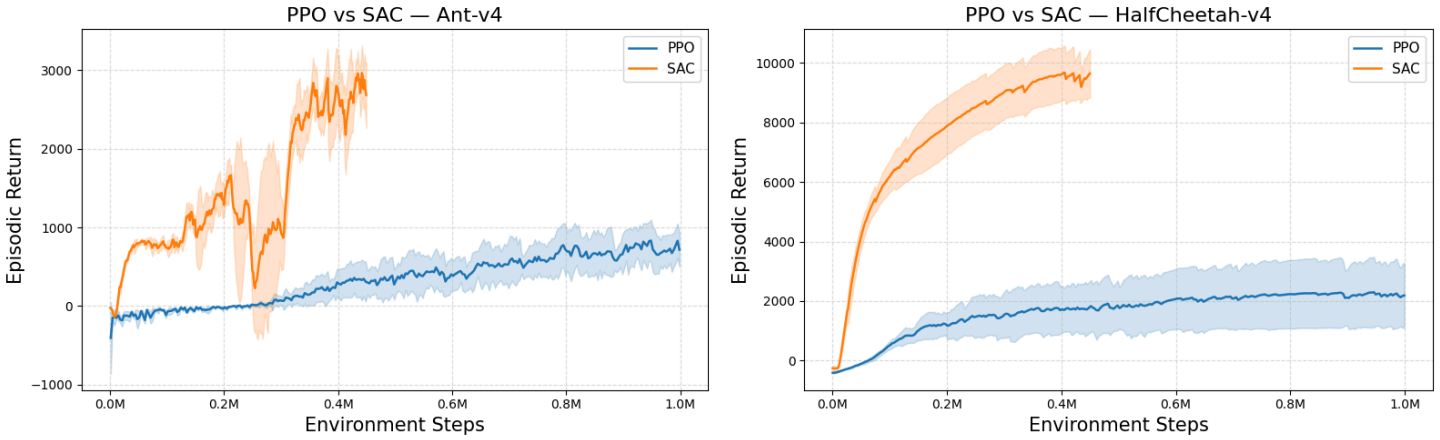

Figure 1 : Learning curves for PPO (blue) and SAC (orange) with shaded 95% confidence intervals across seeds $= \{1, 2, 3\}$. Left: Ant-v4. Right: HalfCheetah-v4.

In terms of training stability and reliability across seeds, PPO maintains a consistently narrow confidence band throughout training in both environments, indicating that its clipped objective produces highly reproducible learning trajectories across seeds. SAC exhibits wider variance, particularly in the 0.2M–0.5M range in Ant-v4, characteristic of its continuous actor-critic feedback loop where the actor, critic and target network are all updating against mutually moving targets. PPO avoids this entirely as its critic is only used to compute advantages for the current batch. Neither algorithm has fully converged by the end of training.

Despite fewer timesteps, SAC required more wall-clock time (2hr vs 1hr on HalfCheetah, 1hr 40min vs 1hr 10min on Ant) due to heavier per-step computation: replay buffer sampling and multiple network updates versus PPO's amortised batch updates. SAC is preferable when environment interactions are the bottleneck; PPO is preferable when fast simulation makes data collection cheap.

In HalfCheetah-v4, both policies learn a broadly similar running strategy, however qualitative differences are apparent when observing the renders side by side. The SAC policy produces notably faster locomotion with fluid, coordinated motion much like a real animal. The PPO policy, while functional, exhibits occasional erratic jumps and lacks the smoothness of SAC's gait, consistent with the lower episodic returns seen in the learning curves.

The Ant policies are more peculiar. Instead of using all four limbs, SAC's policy uses two limbs to stabilise and two to push forward in a rowing motion. PPO learns a more cohesive use of all limbs but still achieves slower locomotion. Notably, both policies actively correct their body orientation relative to the direction of motion, maintaining a preferred heading rather than moving sideways.

For HalfCheetah, both algorithms adequately solve the task. Both achieve fast directed locomotion, with SAC producing notably more fluid motion than PPO's occasionally erratic motion. For Ant, adequate task completion is less convincing. While both achieve forward locomotion, the learned policies are mechanically peculiar. SAC's two-limb rowing motion and PPO's uncoordinated gait both feel far from a natural solution. SAC in particular appears to be exploiting the reward function rather than learning the intended behaviour, finding a local optimum that satisfies the objective while ignoring two limbs entirely. This suggests both algorithms, given the training budgets used, have found reward-maximising shortcuts rather than genuinely solving the task.

7. Proposed Robotics Task

The 2026 Formula 1 regulations introduced a fundamental shift in power unit architecture: roughly half the car's power now comes from electrical deployment via the upgraded MGU-K (tripled in capacity to 350kW), and the MGU-H is removed entirely [5], [6]. However, battery capacity is still the same 4MJ, meaning the battery is charged and depleted multiple times per lap. This makes within-lap energy management a continuous, high-frequency control problem: the driver must constantly decide when to deploy battery energy, when to harvest it (super clipping), and how to coordinate this with the new active aerodynamics, since lifting off disables the active aero allowing more harvesting but reducing straight-line speed. We propose framing this within-lap energy and pace management task as a POMDP, because most variables critical to decision making are hidden from the agent. Tyre degradation is only indirectly observed through surface temperature, yet internal rubber state can collapse suddenly (the "tyre cliff"). Competitor energy levels and strategy are invisible — the agent can see lap times and gaps but not battery state or pit strategy. Track grip evolves each lap as rubber is laid down but cannot be measured directly, and weather conditions affect grip levels. Battery health degrades over a race distance in ways that cannot be fully captured.

The true state s would include the car's exact position, velocity, battery state, tyre condition, fuel load, aerodynamic state, track grip, weather effects, and the hidden state of rival cars. The observation o would consist of car telemetry available to engineers and driver: speed, throttle and brake traces, battery charge, tyre temperature and pressures, lap and sector times, time gaps to nearby cars, overtake availability, active aero deployments, and noisy weather forecasts. The action $a \in \mathbb{R}^3$ is continuous: an energy deployment level (harvest to full deploy), a target pace delta (push vs. conserve), and harvesting intensity (trading lap time for energy, which also disables active aero). The reward function combines sector times relative to target pace, an energy management penalty for depleting the battery at critical moments (e.g. end of straights), and a tyre preservation term.

We would choose SAC for this task. Our experiments showed that high γ was the dominant factor for SAC's performance. When $\gamma = 0.90$, the agent could "see" about 10 steps into the future ($1/(1-\gamma) \approx 10$) and performance was stuck around 2300 regardless of τ. When $\gamma = 0.99$ (effective horizon of 100 steps) performance jumped to 8500–9400. The agent needed to reason about long chains of cause and effect, and locomotion requires these long horizons. Within-lap energy management involves dozens of coupled deployment decisions per lap whose consequences compound over subsequent sections and laps — a deployment burst to overtake now might leave you vulnerable a few corners later, and a competitor within one second will have additional power to deploy as well.

SAC's entropy-regularised objective suits this task because there is no single correct deployment strategy: the optimal profile depends on competitor behaviour, tyre state, and track position, all of which are uncertain, so a stochastic policy that maintains multiple viable strategies is more robust than a single deterministic one. PPO would only be attractive with access to massively parallel simulation, whereas high-fidelity race simulation involving tyre thermodynamics, aero mode interactions, and competitor AI is too expensive per rollout for on-policy data gathering.

Key challenges with this approach are the sim-to-real gap, multi-agent non-stationarity, and reward shaping across competing objectives. Tyre models are notoriously unreliable — even Pirelli's internal models often misjudge degradation speed — so a policy trained in simulation may behave poorly on track where rubber behaviour diverges from the model. Competitor strategies are adversarial and partially observable, yet SAC assumes a stationary environment; one mitigation is to treat opponents as part of the environment and retrain periodically as their strategies evolve. Optimising race position, lap time, energy efficiency, and tyre preservation simultaneously creates a multi-objective reward that requires careful shaping to avoid the agent collapsing onto a single objective.

That said, RL may not be the ideal approach in isolation. Classical optimisation techniques are excellent when you have a good model and well-defined constraints. F1 teams already use these for pit-stop strategies — timing is a relatively low-frequency decision (2–3 times per race), the option space is discrete and enumerable (pit on lap 15, 16, 17… with soft, medium, or hard tyres), and the models, though still imperfect, are good enough at this level of abstraction. But classical techniques struggle with the high-frequency, continuous, reactive decision making that the 2026 regulations demand. "How much energy should I deploy on the second straight given that the car behind just closed to within 1.2 seconds and they have a 3-lap tyre advantage?" This is not a question that can be pre-computed: the state space is too large, decisions are too frequent, and the interaction between energy, tyres, and competitor behaviour is too complex for hand-crafted heuristics to capture optimally. This is exactly where RL excels: learning a reactive policy through experience that maps high-dimensional observations to continuous action spaces. A hybrid approach bridges this boundary — classical methods for strategic decisions, with an RL policy handling continuous within-lap energy deployment, initialised via Imitation Learning from historical race data so the policy begins by replicating expert driver behaviour rather than exploring randomly.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *CoRR*, 2017, [Online]. Available: http://arxiv.org/abs/1707.06347

[2] T. Haarnoja *et al.*, “Soft Actor-Critic Algorithms and Applications,” *CoRR*, 2018, [Online]. Available: http://arxiv.org/abs/1812.05905

[3] M. Towers *et al.*, “Gymnasium: A Standard Interface for Reinforcement Learning Environments.” [Online]. Available: https://arxiv.org/abs/2407.17032

[4] M. Towers *et al.*, “Gymnasium: A Standard Interface for Reinforcement Learning Environments.” [Online]. Available: https://arxiv.org/abs/2407.17032

[5] Fédération Internationale de l'Automobile, “2026 Formula 1 Power Unit Technical Regulations,” technical report, 2024. [Online]. Available: https://www.fia.com/sites/default/files/fia_2026_formula_1_technical_regulations_pu_-_issue_7_-_2024-06-11_1.pdf

[6] Formula 1, “2026 Regulations Explained: All You Need to Know About F1's New Power Units.” 2026.